

---

# Reflexion: Language Agents with Verbal Reinforcement Learning

---

**Noah Shinn**

Northeastern University  
noahshinn024@gmail.com

**Federico Cassano**

Northeastern University  
cassano.f@northeastern.edu

**Edward Berman**

Northeastern University  
berman.ed@northeastern.edu

**Ashwin Gopinath**

Massachusetts Institute of Technology  
agopi@mit.edu

**Karthik Narasimhan**

Princeton University  
karthikn@princeton.edu

**Shunyu Yao**

Princeton University  
shunyuy@princeton.edu

## Abstract

Large language models (LLMs) have been increasingly used to interact with external environments (e.g., games, compilers, APIs) as goal-driven agents. However, it remains challenging for these language agents to quickly and efficiently learn from trial-and-error as traditional reinforcement learning methods require extensive training samples and expensive model fine-tuning. We propose *Reflexion*, a novel framework to reinforce language agents not by updating weights, but instead through linguistic feedback. Concretely, Reflexion agents verbally reflect on task feedback signals, then maintain their own reflective text in an episodic memory buffer to induce better decision-making in subsequent trials. Reflexion is flexible enough to incorporate various types (scalar values or free-form language) and sources (external or internally simulated) of feedback signals, and obtains significant improvements over a baseline agent across diverse tasks (sequential decision-making, coding, language reasoning). For example, Reflexion achieves a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the previous state-of-the-art GPT-4 that achieves 80%. We also conduct ablation and analysis studies using different feedback signals, feedback incorporation methods, and agent types, and provide insights into how they affect performance. We release all code, demos, and datasets at <https://github.com/noahshinn024/reflexion>.

## 1 Introduction

Recent works such as ReAct [30], SayCan [1], Toolformer [22], HuggingGPT [23], generative agents [19], and WebGPT [17] have demonstrated the feasibility of autonomous decision-making agents that are built on top of a large language model (LLM) core. These methods use LLMs to generate text and ‘actions’ that can be used in API calls and executed in an environment. Since they rely on massive models with an enormous number of parameters, such approaches have been so far limited to using in-context examples as a way of teaching the agents, since more traditional optimization schemes like reinforcement learning with gradient descent require substantial amounts of compute and time.

In this paper, we propose an alternative approach called *Reflexion* that uses verbal reinforcement to help agents learn from prior failings. Reflexion converts binary or scalar feedback from the environment into verbal feedback in the form of a textual summary, which is then added as additional context for the LLM agent in the next episode. This self-reflective feedback acts as a ‘semantic’ gradient signal by providing the agent with a concrete direction to improve upon, helping it learn from prior mistakes to perform better on the task. This is akin to how humans iteratively learn to accomplish complex tasks in a few-shot manner – by reflecting on their previous failures in order to form an improved plan of attack for the next attempt. For example, in figure 1, a Reflexion agent learns to optimize its own behavior to solve decision-making, programming, and reasoning tasks through trial, error, and self-reflection.

Generating useful reflective feedback is challenging since it requires a good understanding of where the model made mistakes (i.e. the credit assignment problem [25]) as well as the ability to generate a summary containing actionable insights for improvement. We explore three ways for doing this – simple binary environment feedback, pre-defined heuristics for common failure cases, and self-evaluation such as binary classification using LLMs (decision-making) or self-written unit tests (programming). In all implementations, the evaluation signal is amplified to natural language experience summaries which can be stored in long-term memory.

Reflexion has several advantages compared to more traditional RL approaches like policy or value-based learning: 1) it is lightweight and doesn’t require finetuning the LLM, 2) it allows for more nuanced forms of feedback (e.g. targeted changes in actions), compared to scalar or vector rewards that are challenging to perform accurate credit assignment with, 3) it allows for a more explicit and interpretable form of episodic memory over prior experiences, and 4) it provides more explicit hints for actions in future episodes. At the same time, it does have the disadvantages of relying on the power of the LLM’s self-evaluation capabilities (or heuristics) and not having a formal guarantee for success. However, as LLM capabilities improve, we only expect this paradigm to get better over time.

We perform experiments on (1) decision-making tasks to test sequential action choices over long trajectories, (2) reasoning tasks to test knowledge-intensive, single-step generation improvement, and (3) programming tasks to teach the agent to effectively use external tools such as compilers and interpreters. Across all three types of tasks, we observe Reflexion agents are better decision-makers, reasoners, and programmers. More concretely, Reflexion agents improve on decision-making AlfWorld [24] tasks over strong baseline approaches by an absolute 22% in 12 iterative learning steps, and on reasoning questions in HotPotQA [28] by 20%, and Python programming tasks on HumanEval [6] by as much as 11%.

To summarize, our contributions are the following:

- We propose Reflexion, a new paradigm for ‘verbal’ reinforcement that parameterizes a policy as an agent’s memory encoding paired with a choice of LLM parameters.
- We explore this emergent property of *self-reflection* in LLMs and empirically show that self-reflection is extremely useful to learn complex tasks over a handful of trials.
- We introduce LeetCodeHardGym, a code-generation RL gym environment consisting of 40 challenging LeetCode questions (‘hard-level’) in 19 programming languages.
- We show that Reflexion achieves improvements over strong baselines across several tasks, and achieves state-of-the-art results on various code generation benchmarks.

## 2 Related work

**Reasoning and decision-making** Self-Refine [15] employs an iterative framework for self-refinement to autonomously improve generation through self-evaluation. These self-evaluation and self-improvement steps are conditioned on given task constraints, such as "How can this generation be written in a more positive way". Self-Refine is effective but is limited to single-generation reasoning tasks. Pryzant et al. [21] performs a similar semantic prompt-writing optimization, but is also limited to single-generation tasks. Paul et al. [20] fine-tune critic models to provide intermediate feedback within trajectories to improve reasoning responses. Xie et al. [27] use stochastic beam search over actions to perform a more efficient decision-making search strategy which allows the agent to use foresight advantage due to its self-evaluation component. Yoran et al. [31] and Nair et al.

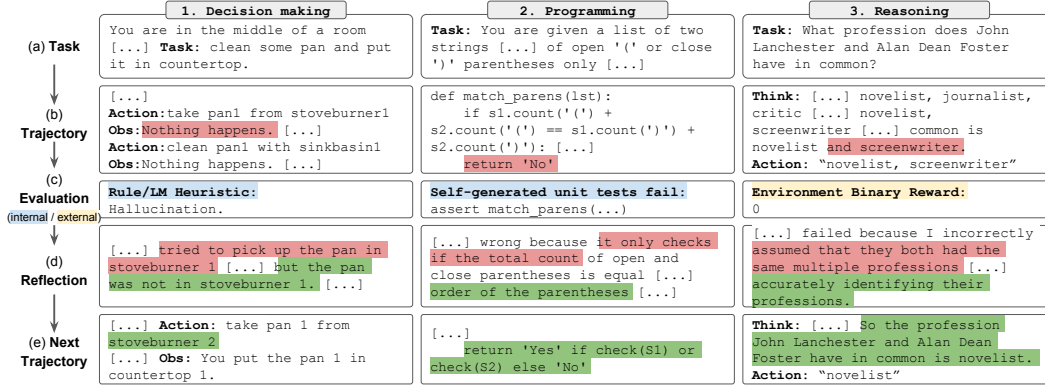


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.

| Related work on reasoning and decision-making |             |                    |                 |               |        |
|-----------------------------------------------|-------------|--------------------|-----------------|---------------|--------|
| Approach                                      | Self refine | Hidden constraints | Decision making | Binary reward | Memory |
| Self-refine [15]                              | ✓           | ✗                  | ✗               | ✗             | ✗      |
| Beam search [27]                              | ✓           | ✓                  | ✓               | ✓             | ✗      |
| <b>Reflexion (ours)</b>                       | ✓           | ✓                  | ✓               | ✓             | ✓      |

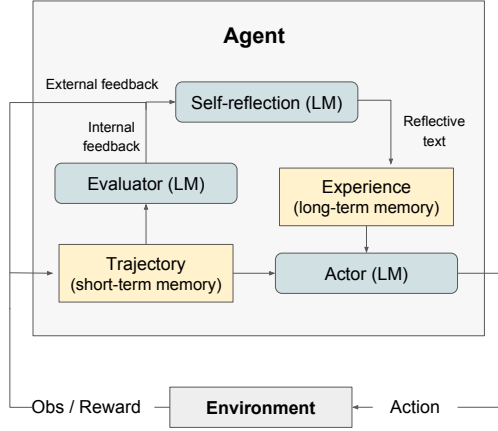
| Related work on programming |                |           |                      |                    |                 |
|-----------------------------|----------------|-----------|----------------------|--------------------|-----------------|
| Approach                    | Test execution | Debugging | Self-generated tests | Multiple languages | Self-reflection |
| AlphaCode [14]              | ✓              | ✗         | ✗                    | ✓                  | ✗               |
| CodeT [5]                   | ✓              | ✗         | ✓                    | ✗                  | ✗               |
| Self-debugging [7]          | ✓              | ✓         | ✗                    | ✗                  | ✗               |
| CodeRL [12]                 | ✓              | ✓         | ✗                    | ✗                  | ✗               |
| <b>Reflexion (ours)</b>     | ✓              | ✓         | ✓                    | ✓                  | ✓               |

[16] use decider models to reason over several generations. Kim et al. [10] use a retry pattern over a fixed number of steps without an evaluation step. Goodman [9] perform a qualitative evaluation step that proposes optimizations to the previous generation. In this paper, we show that several of these concepts can be enhanced with *self-reflection* to build a persisting memory of self-reflective experiences which allows an agent to identify its own errors and self-suggest lessons to learn from its mistakes over time.

**Programming** Several past and recent works employ variations of test-driven development or code debugging practices. AlphaCode [14] evaluates a set of generations on hidden test cases. CodeT [5] uses self-generated unit tests that are used to score generated function implementations. Self-Debugging [7] employs a debugging component that is used to improve existing implementations given feedback from a code execution environment. CodeRL [12] sets the problem in an RL framework using an actor-critic setup to debug programs given feedback from an execution environment. AlphaCode, Self-Debugging and CodeRL are effective in fixing less-complex program bugs, but they rely upon ground truth test cases that invalidate pass@1 eligibility, and do not use self-reflection to bridge the gap between error identification and implementation improvement. CodeT does not access hidden test cases but does not implement a self-learning step to improve code writing.

### 3 Reflexion: reinforcement via verbal reflection

We develop a modular formulation for Reflexion, utilizing three distinct models: an *Actor*, denoted as  $M_a$ , which generates text and actions; an *Evaluator* model, represented by  $M_e$ , that scores the outputs produced by  $M_a$ ; and a *Self-Reflection* model, denoted as  $M_{sr}$ , which generates verbal reinforcement cues to assist the Actor in self-improvement. We provide a detailed description of each of these models and subsequently elucidate their collaborative functioning within the Reflexion framework.




---

**Algorithm 1** Reinforcement via self-reflection

---

```

Initialize Actor, Evaluator, Self-Reflection:
 $M_a, M_e, M_{sr}$ 
Initialize policy  $\pi_\theta(a_i|s_i), \theta = \{M_a, mem\}$ 
Generate initial trajectory using  $\pi_\theta$ 
Evaluate  $\tau_0$  using  $M_e$ 
Generate initial self-reflection  $sr_0$  using  $M_{sr}$ 
Set  $mem \leftarrow [sr_0]$ 
Set  $t = 0$ 
while  $M_e$  not pass or  $t < \text{max trials}$  do
    Generate  $\tau_t = [a_0, o_0, \dots, a_i, o_i]$  using  $\pi_\theta$ 
    Evaluate  $\tau_t$  using  $M_e$ 
    Generate self-reflection  $sr_t$  using  $M_{sr}$ 
    Append  $sr_t$  to  $mem$ 
    Increment  $t$ 
end while
return

```

---

Figure 2: (a) Diagram of Reflexion. (b) Reflexion reinforcement algorithm

**Actor** The Actor is built upon a large language model (LLM) that is specifically prompted to generate the necessary text and actions conditioned on the state observations. Analogous to traditional policy-based RL setups, we sample an action or generation,  $a_t$ , from the current policy  $\pi_\theta$  at time  $t$ , receive an observation from the environment  $o_t$ . We explore various Actor models, including Chain of Thought [26] and ReAct [30]. These diverse generation models allow us to explore different aspects of text and action generation within the Reflexion framework, providing valuable insights into their performance and effectiveness. In addition, we also add a memory component  $mem$  that provides additional context to this agent. This adaption was inspired by Brooks et al. [3], who suggest a policy iteration approach using in-context learning. Details on how this is populated are provided below.

**Evaluator** The Evaluator component of the Reflexion framework plays a crucial role in assessing the quality of the generated outputs produced by the Actor. It takes as input a generated trajectory and computes a reward score that reflects its performance within the given task context. Defining effective value and reward functions that apply to semantic spaces is difficult, so we investigate several variants of the Evaluator model. For reasoning tasks, we explore reward functions based on exact match (EM) grading, ensuring that the generated output aligns closely with the expected solution. In decision-making tasks, we employ pre-defined heuristic functions that are tailored to specific evaluation criteria. Additionally, we experiment with using a different instantiation of an LLM itself as an Evaluator, generating rewards for decision-making and programming tasks. This multi-faceted approach to Evaluator design allows us to examine different strategies for scoring generated outputs, offering insights into their effectiveness and suitability across a range of tasks.

**Self-reflection** The Self-Reflection model instantiated as an LLM, plays a crucial role in the Reflexion framework by generating verbal self-reflections to provide valuable feedback for future trials. Given a sparse reward signal, such as a binary success status (success/fail), the current trajectory, and its persistent memory  $mem$ , the self-reflection model generates nuanced and specific feedback. This feedback, which is more informative than scalar rewards, is then stored in the agent’s memory ( $mem$ ). For instance, in a multi-step decision-making task, when the agent receives a failure signal, it can infer that a specific action  $a_i$  led to subsequent incorrect actions  $a_{i+1}$  and  $a_{i+2}$ . The agent can then verbally state that it should have taken a different action,  $a'_i$ , which would have resulted in  $a'_{i+1}$  and  $a'_{i+2}$ , and store this experience in its memory. In subsequent trials, the agent can leverage its past experiences to adapt its decision-making approach at time  $t$  by choosing action  $a'_i$ . This iterative process of trial, error, self-reflection, and persisting memory enables the agent to rapidly improve its decision-making ability in various environments by utilizing informative feedback signals.

**Memory** Core components of the Reflexion process are the notion of short-term and long-term memory. At inference time, the Actor conditions its decisions on short and long-term memory, similar

to the way that humans remember fine-grain recent details while also recalling distilled important experiences from long-term memory. In the RL setup, the trajectory history serves as the short-term memory while outputs from the Self-Reflection model are stored in long-term memory. These two memory components work together to provide context that is specific but also influenced by lessons learned over several trials, which is a key advantage of Reflexion agents over other LLM action choice works.

**The Reflexion process** Reflexion is formalized as an iterative optimization process in 1. In the first trial, the Actor produces a trajectory  $\tau_0$  by interacting with the environment. The Evaluator then produces a score  $r_0$  which is computed as  $r_t = M_e(\tau_0)$ .  $r_t$  is only a scalar reward for trial  $t$  that improves as task-specific performance increases. After the first trial, to amplify  $r_0$  to a feedback form that can be used for improvement by an LLM, the Self-Reflection model analyzes the set of  $\{\tau_0, r_0\}$  to produce a summary  $sr_0$  which is stored in the memory *mem*.  $sr_t$  is a verbal experience feedback for trial  $t$ . The Actor, Evaluator, and Self-Reflection models work together through trials in a loop until the Evaluator deems  $\tau_t$  to be correct. As mentioned in 3, the memory component of Reflexion is crucial to its effectiveness. After each trial  $t$ ,  $sr_t$  is appended *mem*. In practice, we bound *mem* by a maximum number of stored experiences,  $\Omega$  (usually set to 1-3) to adhere to max context LLM limitations.

## 4 Experiments

We evaluate various natural language RL setups on decision-making, reasoning, and code generation tasks. Specifically, we challenge an agent to perform search-based question answering on HotPotQA [28], multi-step tasks in common household environments in AlfWorld [24], and code writing tasks in competition-like environments with interpreters and compilers in HumanEval [6], MBPP [2], and LeetCodeHard, a new benchmark. Most notably, Reflexion improves performance over strong baselines by 22% in AlfWorld, 20% in HotPotQA, and 11% on HumanEval.

### 4.1 Sequential decision making: ALFWorld

AlfWorld is a suite of text-based environments that challenge an agent to solve multi-step tasks in a variety of interactive environments based on TextWorld [8]. Following Yao et al. [30], we run the agent in 134 AlfWorld environments across six different tasks, including finding hidden objects (e.g., finding a spatula in a drawer), moving objects (e.g., moving a knife to the cutting board), and manipulating objects with other objects (e.g., chilling a tomato in the fridge). We use ReAct [30] as the action generator as Yao et al. [30] has shown success in long trajectory decision-making using explicit intermediate thoughts. AlfWorld tasks naturally require a self-evaluation step as the environment can only signal if a task is complete. To achieve fully autonomous behavior, we implement two self-evaluation techniques: natural language classification using an LLM and a hand-written heuristic. The heuristic is simple: if the agent executes the same action and receives the same response for more than 3 cycles, or if the number of actions taken in the current environment exceeds 30 (inefficient planning), we self-reflect. In the baseline runs, if self-reflection is suggested, we skip the self-reflection process, reset the environment, and start a new trial. In the Reflexion runs, the agent uses self-reflection to find its mistake, update its memory, reset the environment, and start a new trial. To avoid very long prompt windows that may exceed the maximum limit, we truncate the agent’s memory to the last 3 self-reflections (experiences).

To avoid syntactic errors, we provide two domain-specific few-shot trajectories to the agent. We use the same few-shot trajectory examples as Yao et al. [30] with GPT-3 for the LLM. AlfWorld tasks, ReAct few-shot prompts, and Reflexion examples are included in the appendix.

**Results** ReAct + Reflexion significantly outperforms ReAct by completing 130 out of 134 tasks using the simple heuristic to detect hallucinations and inefficient planning. Further, ReAct + Reflexion learns to solve additional tasks by learning in 12 consecutive trials. In the ReAct-only approach, we see that performance increase halts between trials 6 and 7.

**Analysis** A common error in baseline failed AlfWorld trajectories is when an agent thinks that it has possession of an item but does not actually have the item. The agent proceeds to execute several actions in a long trajectory and is not able to backtrack its actions to find the mistake. Reflexion